

Lab 6: EXPLAIN ANALYZE & Query Tuning

(60 min)

Learning Objectives

By the end of this lab you will be able to:

- Read a CockroachDB `EXPLAIN ANALYZE` plan top-to-bottom and identify the dominant cost
- Surface missing-index recommendations from the DB Console **Insights** page
- Apply STORING, partial, composite, and inverted indexes targeted at specific plans
- Watch the optimizer change its mind after `CREATE STATISTICS`
- Use `EXPLAIN ANALYZE (DEBUG)` to produce a downloadable bundle for support
- Diagnose distSQL plans and decide when a query is "as fast as it can get"

Prerequisites

- A 3-node demo cluster
- Familiarity with index types from Lab 4

Setup

```
scripts/crdb up          # start the 3-node cluster (skip if it is already running)
scripts/crdb sql         # open a SQL shell
```

Everything runs in Docker — see [Lab 1](#) for the cluster layout. Use `scripts/crdb.sh` on macOS, Linux, and **WSL** — WSL is Linux, so the shell script is the right one there. Only a native Windows shell (`cmd` or PowerShell) needs `scripts\crdb.bat`.

Load a moderate dataset:

```

CREATE DATABASE catalog;
USE catalog;

CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sku         STRING NOT NULL,
  name        STRING NOT NULL,
  category    STRING NOT NULL,
  price       DECIMAL(10,2) NOT NULL,
  in_stock    BOOL NOT NULL DEFAULT true,
  created     TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE orders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id),
  customer_id UUID NOT NULL,
  qty         INT NOT NULL CHECK (qty > 0),
  total       DECIMAL(12,2) NOT NULL,
  status      STRING NOT NULL,
  placed      TIMESTAMPTZ DEFAULT now()
);

INSERT INTO products (sku, name, category, price)
SELECT
  'SKU-' || lpad(g::STRING, 6, '0'),
  'Product ' || g,
  (ARRAY['books', 'elec', 'toys', 'home', 'garden', 'sport', 'food', 'health', 'auto', 'pet'])[1 + (g
    % 10)],
  (random() * 200)::DECIMAL(10,2)
FROM generate_series(1, 5000) g;

INSERT INTO orders (product_id, customer_id, qty, total, status)
SELECT
  (SELECT id FROM products ORDER BY random() LIMIT 1),
  gen_random_uuid(),
  1 + (random() * 4)::INT,
  (random() * 500)::DECIMAL(12,2),
  (ARRAY['open', 'paid', 'shipped', 'shipped', 'shipped', 'cancelled'])[1 + (random()*5)::INT]
FROM generate_series(1, 100000);

CREATE STATISTICS p FROM products;
CREATE STATISTICS o FROM orders;

```

Tasks

Part A: Profile Three Queries — Predict First, Then Look (15 min)

For each query, **predict** the bottleneck before reading the plan.

1. Q1 — Most popular categories in the last hour:

```

EXPLAIN ANALYZE
SELECT p.category, count(*) AS cnt, sum(o.total) AS revenue
FROM orders o JOIN products p ON o.product_id = p.id
WHERE o.placed > now() - INTERVAL '1 hour'
GROUP BY p.category
ORDER BY revenue DESC;

```

Predict: scan, join, or aggregation? Where does KV time go?

2. Q2 — Recent orders for one customer:

```

EXPLAIN ANALYZE
WITH cust AS (SELECT customer_id FROM orders LIMIT 1)
SELECT id, qty, total, placed
FROM orders, cust
WHERE orders.customer_id = cust.customer_id
ORDER BY placed DESC
LIMIT 20;

```

Predict: with no index on `customer_id`, what does the plan do?

3. Q3 — Count orders by status:

```

EXPLAIN ANALYZE
SELECT status, count(*) FROM orders GROUP BY status;

```

Predict: no good index. Anything clever happen?

Now run them. For each, note:

- The plan tree's top-level **execution time**
- The **distribution** (local / full)
- Whether the executor was **vectorized**
- KV time at scan nodes vs total time

Part B: Add the Right Indexes (15 min)

1. Q2 fix — covering composite:

```

CREATE INDEX orders_by_customer_placed
ON orders(customer_id, placed DESC)
STORING (qty, total);

```

Re-run Q2 EXPLAIN ANALYZE. Confirm: index used, single-key span, no index join, execution time drops.

2. Q1 fix — placed-with-storing, hash-sharded:

```
CREATE INDEX orders_by_placed
ON orders(placed DESC)
USING HASH
STORING (product_id, total)
WITH (bucket_count = 8);
```

Re-run Q1. The scan should now be bounded to the last hour's rows, spread across 8 buckets.

3. Q3 — is an index worth it?

```
CREATE INDEX orders_by_status ON orders(status);

EXPLAIN ANALYZE
SELECT status, count(*) FROM orders GROUP BY status;
```

Did it help? Sometimes index addition is moot — full-table count over a small set of distinct values can still be cheap streaming via the primary key.

Part C: Use the Insights Page (10 min)

1. Trigger an obviously-suboptimal query:

```
SELECT name, price FROM products WHERE sku = 'SKU-001234';
```

No index on `sku`. The DB Console **SQL Activity** → **Insights** page should show:

```
Create index recommendation: CREATE INDEX ON products(sku);
```

2. Apply the recommendation:

```
CREATE INDEX ON products(sku);
```

Re-run the query and refresh Insights — the alert clears.

3. Inspect insights from SQL:

```
SELECT *
FROM crdb_internal.cluster_execution_insights
ORDER BY end_time DESC
LIMIT 5;
```

Part D: Stale Statistics (10 min)

1. Bulk-update a chunk of orders to shift the distribution:

```
UPDATE orders SET status = 'shipped' WHERE status = 'open';
```

2. Run a selectivity-sensitive query:

```
EXPLAIN ANALYZE
SELECT count(*) FROM orders WHERE status = 'open';
```

Compare **estimated** vs **actual** row count. They should be far apart now — the optimizer's stats predate the update.

3. Refresh stats:

```
CREATE STATISTICS o2 FROM orders;
```

4. **Re-run.** The estimate should now match reality. Plans that depend on selectivity (joins especially) improve.

5. Auto-refresh control:

```
SHOW CLUSTER SETTING sql.stats.automatic_collection.enabled;
SHOW CLUSTER SETTING sql.stats.automatic_collection.fraction_stale_rows;
```

Auto-collection fires when fraction-stale-rows exceeds the threshold. Sometimes it's not aggressive enough for bulk imports — call `CREATE STATISTICS` manually after large data movements.

Part E: EXPLAIN ANALYZE (DEBUG) — Bundles for Support (5 min)

When you can't reproduce a slow query, capture a bundle:

```
EXPLAIN ANALYZE (DEBUG)
SELECT p.category, count(*)
FROM orders o JOIN products p ON o.product_id = p.id
WHERE o.placed > now() - INTERVAL '1 hour'
GROUP BY p.category;
```

The output prints a `cockroach demo` command to load the bundle, plus a path to the zip. (That command is CockroachDB's own suggestion; to run it here, use `scripts/crdb run demo --with-load ...` inside the container, or download the bundle from the DB Console and open it on a machine with the binary.) The bundle contains the plan, the schema, statistics, and a small reproducible dataset — perfect for opening a support case or shipping a repro to your team's expert.

Part G: When Is a Query as Fast as It Can Get? (5 min)

A heuristic checklist. If a query checks all boxes, further tuning is unlikely to help much:

- ☐ Plan is **distribution: local** AND scan span is **single-key** OR
- ☐ Plan is **distribution: full** AND it's an aggregate over the whole dataset
- ☐ **Vectorized: true**
- ☐ **KV time** is the bulk of execution time (you're I/O-bound, not compute-bound)
- ☐ Estimated and actual row counts match within ~10×
- ☐ No index joins on hot read paths

- ☐ No full table scans on tables > a few hundred MB

If any of these fail, there's headroom. Walk through them for Q1, Q2, Q3 again with your latest indexes.

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 10 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part F: DistSQL & Vectorization (10 min)

CockroachDB ships two executors: **vectorized** (default, batch-oriented) and **row-by-row** (for unsupported types). And every plan picks a **distribution**: local (single-node) or distributed.

1. A naturally-distributable query:

```
EXPLAIN
SELECT category, count(*) FROM products GROUP BY category;
```

Look for `distribution: full` and `vectorized: true`.

2. A query that has to localize:

```
EXPLAIN
SELECT id, name FROM products WHERE sku = 'SKU-001234';
```

`distribution: local` — a single-row lookup doesn't benefit from distribution.

3. Force the row-by-row engine (rarely useful — for demonstration only):

```
SET vectorize = 'off';
EXPLAIN ANALYZE
SELECT category, count(*) FROM products GROUP BY category;
SET vectorize = 'on'; -- restore the default
```

You may see a slight performance difference (usually vectorized is faster).

Cleanup

```
DROP DATABASE catalog CASCADE;
```

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

Lab 6 Deliverables

☐ **Plans read:** predicted bottlenecks for three queries; confirmed with EXPLAIN ANALYZE
 ☐ **Indexes applied:** covering composite, hash-sharded, single-column
 ☐ **Insights used:** surfaced and applied a missing-index recommendation
 ☐ **Stats refreshed:** observed and corrected an optimizer estimate that drifted
 ☐ **Debug bundle generated:** produced a downloadable EXPLAIN ANALYZE (DEBUG) artifact
 ☐ **DistSQL & vectorization** understood

Challenge Exercises

1. **Hunt the bad estimate.** This query runs "fine" but the optimizer estimates wildly wrong at one node. Find it.

```
EXPLAIN ANALYZE
SELECT p.category, count(*) FILTER (WHERE o.status = 'open') AS open_count
FROM products p LEFT JOIN orders o ON o.product_id = p.id
WHERE p.in_stock = true
GROUP BY p.category;
```

Hint: look for a node where `estimated row count` is off by $\geq 10\times$ from actual.

2. **Equivalent rewrites.** Three forms of "open orders by customer" follow. Which has the best plan?

```
-- A
SELECT o.* FROM orders o WHERE o.status = 'open';

-- B
SELECT o.* FROM orders o JOIN (SELECT 'open'::STRING AS s) x ON o.status = x.s;

-- C
SELECT o.* FROM orders o WHERE EXISTS (SELECT 1 WHERE o.status = 'open');
```

3. **Subquery vs JOIN.** Rewrite this scalar subquery as a JOIN — does the optimizer notice the equivalence?

```
SELECT (SELECT count(*) FROM orders WHERE product_id = p.id) AS cnt, p.name
FROM products p ORDER BY cnt DESC LIMIT 10;
```

Reference

Plan element	Meaning
scan: table@idx, spans: [/x - /x]	Single-key lookup — ideal
scan: table@primary, spans: [-]	Full table scan — avoid on big tables
index join	Per-row PK lookup — add STORING
distribution: full	Cross-node scan — fine for aggregates
distribution: local	Single-node execution — good for point lookups
vectorized: true	Batch execution — default and usually fastest
KV time	Time spent in the storage layer
execution time	Top-line wall clock